

OpenEdu Orchestrator

An Agentic Multi-Agent Data Synchronization System

PIEAS Website → OpenEduCat (Odoo) ERP

Implementation Report – Test Build on Dummy Data

Muhammad Shayan Asif

July 29, 2026

Source report: *PIEAS_OpenEduCat_MultiAgent_Report 2.pdf* (Revision 2, simplified 4-agent design), a Multi-Agent Systems / AI Agents course assignment.

This document: describes a complete, working, and independently testable implementation of that design, built with **LangGraph**, running against **deterministic dummy data** that stands in for the real PIEAS database and a real self-hosted OpenEduCat/Odoo instance.

Repository: <https://github.com/SubhanSandhu312/OpenEdu-Orchestrator>

Result: 4 cooperating agents, 1 optional Validation agent, 1 LangGraph state machine serving 3 sync cycles, 3 independently-owned SQLite stores, and a 57-test pytest suite – all green.

Contents

1	Executive Summary	2
2	Background and Problem Statement	2
2.1	Why this system exists	2
2.2	The constraint that shapes everything	3
3	System Architecture	3
3.1	Design principle: one agent, one clear job	3
3.2	Architecture diagram	3
3.3	Three independently-owned databases	3
4	Agent-by-Agent Breakdown	4
4.1	Illustrative excerpt: the Orchestrator’s classification logic	4
5	The LangGraph Orchestration Layer	5
5.1	Why one graph, not three	5
5.2	Pipeline state machine	6
5.3	A subtlety worth documenting: pagination and running totals	6
6	Change Detection and Deletion Detection	7
6.1	Change detection: timestamp watermark	7
6.2	Deletion detection: full ID list, compared locally	7
6.3	What happens to a detected deletion	7
7	Database Schemas	8
8	Design Decisions Carried Over From the Report	9
9	Refinements Beyond the Report’s Literal Text	9
10	Technology Stack	9
11	Testing and Quality Assurance	10
11.1	Suite structure	10
11.2	Test run output	11
12	Command-Line Interface	12
13	End-to-End Demo Walkthrough	12
13.1	Final state	13
14	Known Characteristics and Limitations	13
15	Path to Production	14
16	Conclusion	14
A	Appendix: Project File Tree	14

1 Executive Summary

This project turns a course-assignment design document into working software. The source report specifies an **agentic, multi-agent system** for moving data one time from the PIEAS website into OpenEduCat (an Odoo-based Student Information System), and then keeping the two in sync forever after – despite PIEAS exposing no API and no webhooks of any kind.

The implementation delivered here is not a diagram-only or pseudocode exercise. It is a runnable Python package that:

- implements all four agents the report specifies (**Extractor**, **Transformer**, **Loader**, **Orchestrator**) plus the optional fifth **Validation Agent**, with the exact non-overlapping responsibilities the report defines;
- wires them together as a single **LangGraph StateGraph** that serves all three operating cycles the report describes: bulk migration, the timestamp-watermark change cycle, and the full-ID-list deletion cycle;
- runs against a **deterministic dummy PIEAS database** (60 students, 18 faculty, 14 courses, generated with a fixed random seed) and a **mock OpenEduCat/Odoo client** that exposes the same method surface (`create`, `write`, `search_read`, `read`) that a real Odoo XML-RPC connection would;
- enforces the report’s strict agent-ownership rules *structurally*, by giving each agent module a connection to exactly one of three separate SQLite databases;
- is verified by a **57-test pytest suite** covering every agent in isolation, every cycle end-to-end (including pagination and idempotency), and a full cross-entity lifecycle scenario; and
- ships a **command-line interface** (`seed`, `migrate`, `sync`, `deletion-check`, `mutate`, `status`, `demo`) that lets anyone watch the whole design work end to end with a single command.

2 Background and Problem Statement

2.1 Why this system exists

PIEAS currently maintains its own website/system holding student, faculty, and course data. The institution wants to adopt **OpenEduCat** – a free, open-source, LGPL-3 Student Information System built as a set of modules on top of **Odoo** – as its system of record going forward. Odoo is a modular ERP with a Python backend and a PostgreSQL database whose Object-Relational Mapping (ORM) layer enforces validation, computed fields, and related-record book-keeping on every write; bypassing it (writing straight to PostgreSQL) risks silent, half-formed data with no error raised.

This creates two distinct problems, both of which the source report frames as requiring an *agentic* solution rather than a single script:

1. **One-time migration** – every existing PIEAS record must be transferred into OpenEduCat before it can become the system of record.
2. **Ongoing synchronization** – after migration, PIEAS keeps being used. New students are admitted, records are edited, and some are removed. OpenEduCat must reflect all of this automatically.

2.2 The constraint that shapes everything

“PIEAS exposes no API and no webhooks. It cannot notify anything when its data changes. Any detection of change, and any detection of deletion, has to happen by actively checking PIEAS, not by PIEAS pushing updates out.”

This single constraint is why the system needs a scheduler-driven, stateful comparison loop (the Orchestrator) rather than an event-driven integration – and why detecting a *deletion* turns out to require an entirely different query shape than detecting a *change* (Section 6).

3 System Architecture

3.1 Design principle: one agent, one clear job

The report considered, and deliberately rejected, a fifth “Monitor” agent dedicated to change detection – its job (deciding when to check PIEAS, comparing results against known state) overlapped entirely with the Orchestrator’s, since the Orchestrator already owns the state needed to do that comparison. The final design therefore has four agents with strictly non-overlapping responsibilities:

- **Only the Extractor** ever talks to PIEAS. It is a pure fetcher with no memory between calls and no opinion about what the data means.
- **Only the Orchestrator** makes decisions. It is the only agent that reads or writes the local state store.
- **Transformer and Loader are stateless workers.** Each does exactly one job on whatever it is handed, regardless of which cycle triggered it.

3.2 Architecture diagram

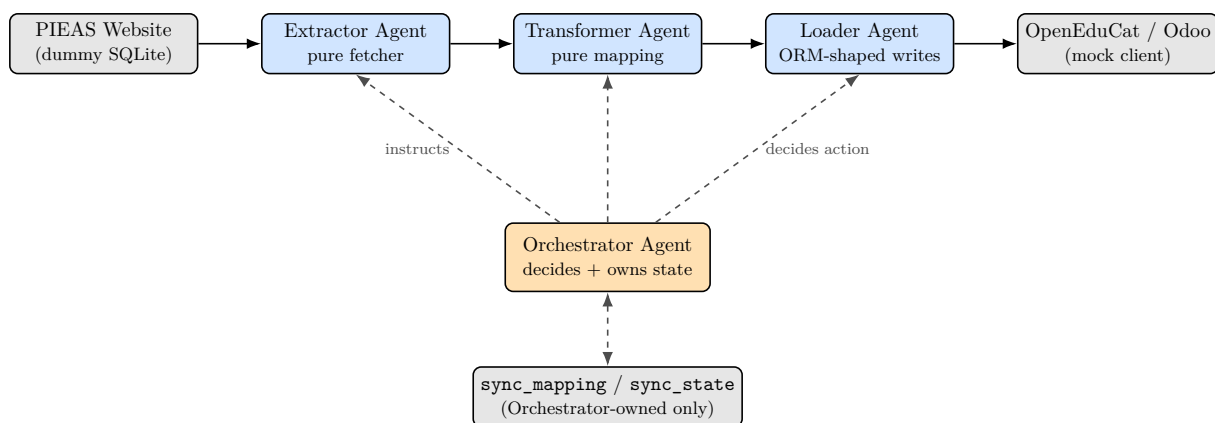


Figure 1: Four-agent architecture. Solid arrows are the data pipeline (PIEAS → OpenEduCat); dashed arrows are instructions and state ownership, all controlled by the Orchestrator – the only agent with memory of prior syncs.

3.3 Three independently-owned databases

The separation of concerns above is enforced *structurally*, not just by convention: three separate SQLite files stand in for three separate real systems, and only one agent module has a code path to open each one.

Database file	Stands in for	Only opened by
data/pieas.db	The PIEAS website's database	ExtractorAgent
data/openeducat_mock.db	Self-hosted Odoo + OpenEduCat	LoaderAgent (writes), ValidationAgent (reads)
data/orchestrator_state.db	N/A – exists only for this system	OrchestratorAgent

Table 1: Database ownership boundaries.

4 Agent-by-Agent Breakdown

Agent	Responsibility	Module
Extractor	Pure fetcher; the only agent that ever contacts PIEAS. Dispatches on an instruction: <code>fetch_changed</code> (watermarked rows), <code>fetch_page</code> (paginated full pull), or <code>fetch_ids</code> (bare ID list for deletion detection). Holds no state between calls.	<code>extractor.py</code>
Transformer	Maps a PIEAS-shaped record onto an OpenEduCat-shaped record, per entity type (<code>op.student</code> , <code>op.faculty</code> , <code>op.course</code>). A pure function: identical input always produces identical output, with no I/O and no awareness of create vs. update.	<code>transformer.py</code>
Loader	Writes into OpenEduCat via an ORM-shaped client. Executes exactly the action it is told: <code>create</code> , <code>update</code> (write), or <code>archive</code> (write <code>active=False</code>). Makes no decisions of its own.	<code>loader.py</code>
Orchestrator	The only decision-making agent. Owns and is the sole reader/writer of <code>sync_mapping</code> and <code>sync_state</code> . Decides what to fetch, classifies every record as <code>create/update/unchanged</code> , and (in the deletion cycle) <code>archive</code> . Owns the switch from BULK mode to SYNC mode.	<code>orchestrator.py</code>
Validator (<i>optional</i>)	Re-reads a record after a Loader write and confirms it matches what was sent. Purely observational – never blocks or retries a write itself; findings are advisory and appended to the run report.	<code>validator.py</code>

Table 2: Agent responsibilities, mirroring the source report's Section 3.3 / 3.4 exactly.

4.1 Illustrative excerpt: the Orchestrator's classification logic

This is the heart of the decision-making agent – the code that decides `create` vs. `update` vs. `unchanged` for every fetched PIEAS row, by looking up the mapping table and comparing a content hash:

Listing 1: Orchestrator record classification (`agents/orchestrator.py`).

```

1 def classify_records(self, entity_type, records):
2     classified = []
3     for record in records:
4         pieas_id = record["pieas_id"]
5         business_fields = {k: v for k, v in record.items() if k != "
            last_updated"}
6         h = store.content_hash(business_fields)

```

```
7 mapping = store.get_mapping(self._conn, pieas_id, entity_type)
8 if mapping is None:
9     action, openeducat_id = "create", None
10 elif mapping["content_hash"] == h:
11     action, openeducat_id = "unchanged", mapping["openeducat_id"]
12 else:
13     action, openeducat_id = "update", mapping["openeducat_id"]
14 classified.append({
15     "entity_type": entity_type, "pieas_id": pieas_id, "action": action
16     ,
17     "source_record": record, "openeducat_id": openeducat_id, "
18     content_hash": h,
19 })
20 return classified
```

5 The LangGraph Orchestration Layer

5.1 Why one graph, not three

Bulk migration, the change cycle, and the deletion cycle all share the same Extractor → Transformer → Loader pipeline; they differ only in what the Orchestrator asks the Extractor to fetch and what it does with the result. Rather than building three separate LangGraph graphs, the implementation builds **one StateGraph** whose nodes branch on a `mode` field ("bulk", "change", or "deletion") carried in the graph state. This keeps the report's cycles honest about sharing a pipeline, instead of accidentally re-implementing three near-identical graphs.

5.2 Pipeline state machine

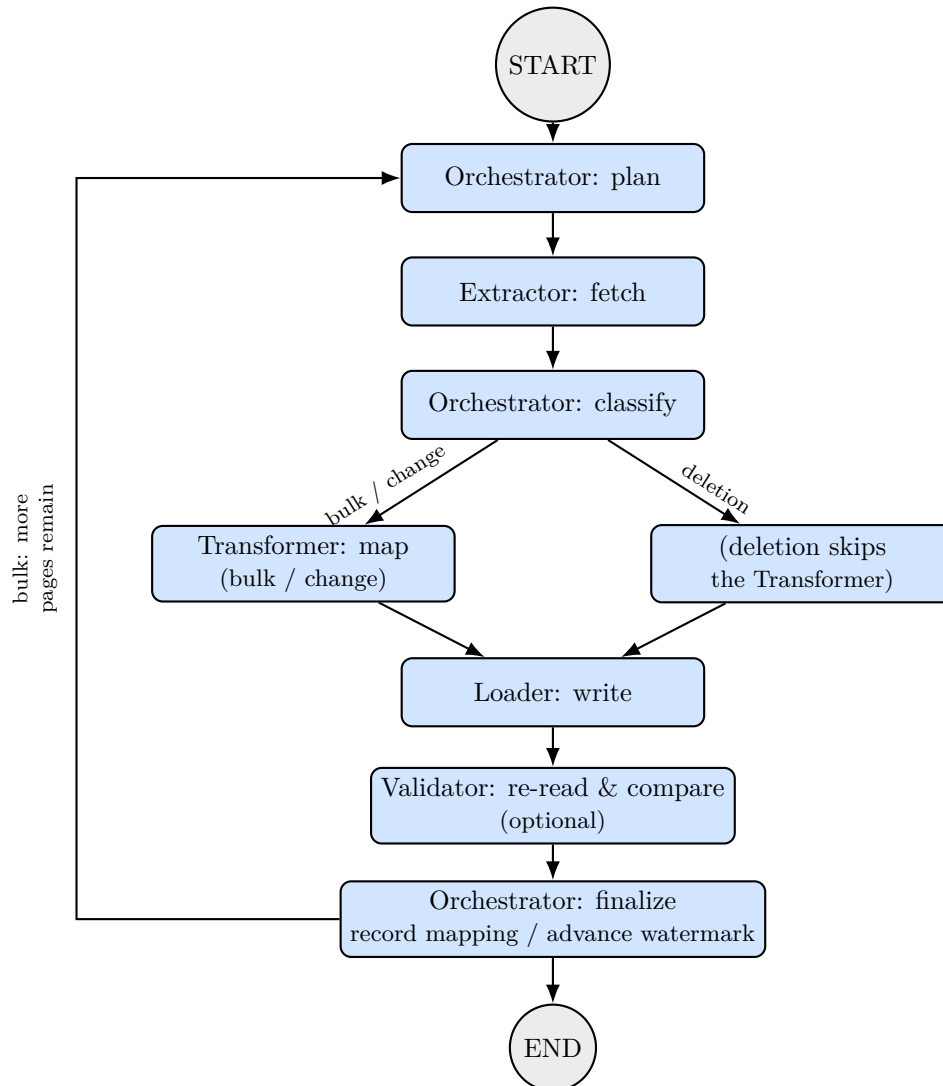


Figure 2: The shared LangGraph `StateGraph`. `mode` decides branching at *classify* (whether the Transformer is invoked) and at *finalize* (whether bulk migration loops back for another page or the run ends).

5.3 A subtlety worth documenting: pagination and running totals

LangGraph state keys are last-write-wins by default (no reducer was declared for the count fields). Bulk migration loops *plan* → *finalize* once per page, so a naive implementation that read `classified/load_results` directly from the *final* graph state after the whole run would only see the **last page’s** numbers – silently under-reporting a multi-page bulk migration’s totals. This was caught during integration testing (Section 11) and fixed by carrying running totals (`total_fetched`, `total_created`, ...) inside the state itself, with each pass through *finalize* adding that page’s counts onto the total already carried forward:

Listing 2: Running-total accumulation across bulk-migration pages (`graph.py`).

```

1 update = {
2     "total_fetched": state.get("total_fetched", 0) + page_fetched,
3     "total_created": state.get("total_created", 0) + page_created,
4     "total_updated": state.get("total_updated", 0) + page_updated,
5     "total_unchanged": state.get("total_unchanged", 0) + page_unchanged,

```

```

6     "total_archived": state.get("total_archived", 0) + page_archived,
7 }
8 if state["mode"] == "bulk" and state.get("more_pages"):
9     update["page_offset"] = state.get("page_offset", 0) + state.get("page_size", BULK_PAGE_SIZE)
10 return update

```

6 Change Detection and Deletion Detection

6.1 Change detection: timestamp watermark

The report considered and rejected a boolean “dirty flag” column reset after each sync, on the grounds that resetting it blindly can silently drop a second change that arrives while a row is mid-sync (a race condition). The adopted method instead keeps a `last_updated` timestamp per row and a single `last_synced_at` watermark the Orchestrator owns:

Listing 3: Change-cycle query (adapted per entity table).

```

1 SELECT * FROM students
2 WHERE last_updated > :watermark
3 ORDER BY last_updated ASC;

```

This implementation adds one refinement beyond the report’s literal text: the watermark advances to the **maximum `last_updated` actually processed in the run**, not to wall-clock `NOW()`. This is safer against a row changing again while a run is in progress than the report’s literal `UPDATE sync_state SET last_synced_at = NOW()`.

6.2 Deletion detection: full ID list, compared locally

A filtered `last_updated` query can never see a deletion – a row that no longer exists is simply never returned. Detecting a deletion therefore requires a structurally different, less-frequent check:

1. The Extractor fetches the **full, unfiltered** list of PIEAS IDs (IDs only, to keep the pull cheap).
2. The Orchestrator compares that list against its own `sync_mapping` table – **locally**; PIEAS is not queried again. Any mapping row whose `pieas_id` is absent from the fetched list is a detected deletion.

The report expresses this comparison as a `LEFT JOIN ... WHERE ... IS NULL` against a live database table. Because `sync_mapping` and the PIEAS table live in genuinely separate SQLite files in this build, the equivalent comparison is done as a Python set difference over the two locally-available lists – semantically identical to the report’s `LEFT JOIN`, without requiring a fragile cross-database `ATTACH`.

6.3 What happens to a detected deletion

Per the report’s Section 5.4, the adopted handling is to **archive**, not hard-delete: the Loader issues `write(model, id, {"active": False})`. The record disappears from normal OpenEduCat views but its history (grades, past enrollment) is preserved, and the action is reversible if a “deletion” on PIEAS was actually a rename.

A known, intentional characteristic

Once a record is permanently gone from PIEAS, *every* subsequent deletion cycle will still find its mapping row and re-issue the archive write, since nothing in the report’s schema marks a mapping as “already handled.” This is harmless – archiving an already-inactive record is a no-op against real Odoo too – but it does mean the deletion cycle’s reported **archived** count means “archive calls issued,” not strictly “newly discovered deletions.” This was a deliberate choice to avoid inventing a field the source report does not specify.

7 Database Schemas

Table	Key columns	Entity
students	pieas_id (PK), roll_number, first/last_name, email, gender, date_of_birth, department, batch_year, last_updated	student
faculty	pieas_id (PK), employee_code, first/last_name, email, department, designation, last_updated	faculty
courses	pieas_id (PK), code, name, department, credit_hours, semester, last_updated	course

Table 3: Dummy PIEAS schema. A trigger on each table bumps **last_updated** automatically whenever an **UPDATE** does not set it explicitly, exactly mirroring how a real database’s auto-maintained timestamp column behaves.

Table	Key columns	Odoo model
op_student	id (PK), pieas_id, name, email, gender, birth_date, department, batch_year, active, create/write_date	op.student
op_faculty	id (PK), pieas_id, name, email, department, designation, active, create/write_date	op.faculty
op_course	id (PK), pieas_id, code, name, department, credit_hours, semester, active, create/write_date	op.course

Table 4: Mock OpenEduCat schema – deliberately mirrors real Odoo conventions (**active**, **write_date**).

Table (Orchestrator-owned)	Columns
sync_mapping	id (PK), pieas_id, openeducat_id, entity_type, content_hash, last_synced_at – UNIQUE(pieas_id, entity_type)
sync_state	entity_type (PK), last_synced_at

Table 5: Orchestrator’s private state store – taken directly from the report’s Listing 4, with **content_hash** used as a bonus refinement (Section 9).

8 Design Decisions Carried Over From the Report

Decision point	Resolution (as implemented)
Change detection mechanism	Timestamp watermark, not a boolean dirty flag – avoids the reset race condition.
Deletion detection mechanism	Separate, less-frequent cycle: full ID fetch, compared locally against <code>sync_mapping</code> .
What to do with a detected deletion	Archive (<code>active=False</code>), not hard-delete – reversible, preserves history.
How to write into the target	Through an ORM-shaped interface (<code>create/write</code>), never raw SQL – preserves validation and computed-field behaviour.
Was a separate Monitor Agent needed?	No – its responsibilities are absorbed by the Orchestrator, which already owns the state needed for the comparison.

Table 6: Every major design fork in the source report, and how this implementation resolves it – unchanged from the report’s own conclusions.

9 Refinements Beyond the Report’s Literal Text

Two small engineering decisions go slightly beyond what the report specifies verbatim, both documented in code comments and in the project’s `README.md` so they are never mistaken for the report’s own design:

1. **Watermark advances by processed maximum, not `NOW()`.** Safer against a row changing again mid-run than the report’s literal Listing 7.
2. **A content-hash check layered on top of the watermark.** A record whose `last_updated` was touched but whose business fields are byte-for-byte identical classifies as `unchanged` rather than `update`. This also lets **bulk migration seed the watermark itself**, so the very first change cycle afterwards does not have to re-fetch and re-classify every just-migrated row only to discover where the watermark should start.

10 Technology Stack

Library	Role	Why
<code>langgraph</code>	Orchestration graph / state machine	Purpose-built for exactly this kind of stateful, branching agent pipeline; not restricted to LLM-driven agents.
<code>pydantic</code>	Typed data models	Malformed records fail loudly at a model boundary instead of silently corrupting a write.
<code>sqlite3</code> (stdlib)	Three data stores	Zero external services needed to run or test the whole system; trivially swappable for a real driver later.
<code>Faker</code>	Deterministic dummy data	Reproducible fake PIEAS records with a fixed seed.
<code>click</code>	CLI	Clean subcommands (<code>seed</code> , <code>migrate</code> , <code>sync</code> , ...).
<code>rich</code>	CLI output formatting	Readable tables/console output for the <code>status</code> and <code>demo</code> commands.
<code>pytest</code>	Test suite	57 tests, fixtures for isolated per-test databases via <code>tmp_path</code> .

Table 7: Full dependency list (`requirements.txt`).

Note on “agentic”: no LLM calls are made anywhere in this pipeline. The system is agentic in the report’s own sense – autonomous, cooperating components with clearly separated responsibilities and a decision-making Orchestrator – implemented deterministically so that every run is reproducible and testable byte-for-byte. LangGraph was chosen specifically because it models this kind of stateful multi-step orchestration regardless of whether any step calls an LLM; nothing about the design would change if a future extension (e.g., an LLM-assisted Transformer for messy/ambiguous field mapping) were added on top.

11 Testing and Quality Assurance

11.1 Suite structure

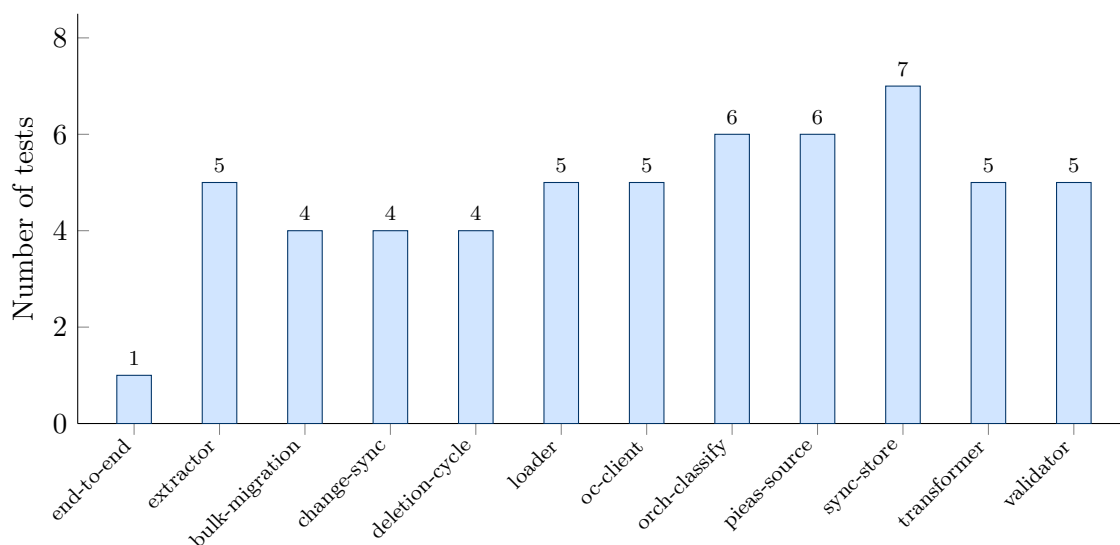


Figure 3: Distribution of the 57-test pytest suite across test modules.

Test file	What it verifies
test_pieas_source.py	Schema, seeding, pagination, watermark filtering, trigger-based timestamp bump, real row deletion.
test_openeducat_client.py	create/write/archive, domain filtering, unknown-model error handling.
test_sync_store.py	Upsert semantics, per-entity scoping, watermark read/write, content-hash stability.
test_extractor.py	Instruction dispatch, statelessness across calls.
test_transformer.py	Per-entity field mapping, purity, unknown-entity error.
test_loader.py	Create/update/archive execution, graceful failure on a bad update.
test_orchestrator_classification.py	create/update/unchanged classification, deletion-candidate detection, per-entity scoping, bulk-mode flip.
test_validator.py	Field-match and mismatch detection, archive-state verification.
test_graph_bulk_migration.py	Multi-page pagination totals, per-entity isolation, re-run idempotency.
test_graph_change_sync.py	Edit + new-admission detection, no-op reruns, watermark monotonicity, cross-entity isolation.
test_graph_deletion_cycle.py	Archival correctness, and the structural proof that the change cycle <i>cannot</i> see a deletion.
test_end_to_end.py	Full lifecycle across all three entity types in one scenario.

Table 8: Test file responsibilities.

11.2 Test run output

Listing 4: Actual output of the full suite.

```

1 $ python -m pytest -v
2 ===== test session starts
3 collected 57 items
4
5 tests/test_end_to_end.py::test_full_lifecycle_across_all_entity_types
6 PASSED
7 tests/test_extractor.py ..... 5 passed
8 tests/test_graph_bulk_migration.py ..... 4 passed
9 tests/test_graph_change_sync.py ..... 4 passed
10 tests/test_graph_deletion_cycle.py ..... 4 passed
11 tests/test_loader.py ..... 5 passed
12 tests/test_openeducat_client.py ..... 5 passed
13 tests/test_orchestrator_classification.py ..... 6 passed
14 tests/test_pieas_source.py ..... 6 passed
15 tests/test_sync_store.py ..... 7 passed
16 tests/test_transformer.py ..... 5 passed
17 tests/test_validator.py ..... 5 passed
18 ===== 57 passed in 2.93s
=====

```

12 Command-Line Interface

Command	Purpose
<code>seed</code>	(Re)create the dummy PIEAS database with fake data; optionally wipes OpenEduCat + the state store too.
<code>migrate -entity all</code>	Run the one-time bulk migration cycle (skips an entity already past BULK mode).
<code>mutate -entity student -update N -insert N -delete N</code>	Simulate PIEAS “still being used”: edits, new admissions, removals.
<code>sync -entity all [-loop -interval S]</code>	Run the change-detection cycle once, or keep running on a schedule.
<code>deletion-check -entity all status</code>	Run the deletion-detection cycle. Print row counts, active/archived counts, mapping counts, and watermarks across all three databases.
<code>demo</code>	One command that walks through the entire lifecycle: seed → bulk migrate → mutate → change cycle → deletion cycle → idempotency re-run → status.

Table 9: `python -m openedu_orchestrator` command reference (each run as a subcommand, e.g. `python -m openedu_orchestrator seed`).

13 End-to-End Demo Walkthrough

Running `python -m openedu_orchestrator demo` exercises every part of the design in sequence, against freshly-seeded data (60 students, 18 faculty, 14 courses):

1. **Seed** – dummy PIEAS populated deterministically (seed = 42).
2. **Bulk migration** – all 60/18/14 records fetched (student migration spans 3 pages at page size 25), transformed, and created in the mock OpenEduCat, with the watermark seeded for each entity type.
3. **Simulate PIEAS activity** – a handful of edits, new admissions, and removals across all three entity types.
4. **Change cycle** – picks up exactly the edited and newly-admitted rows (never the removed ones – structurally invisible to a watermark query).
5. **Deletion cycle** – fetches the full current ID list and archives exactly the rows whose mapping no longer has a matching PIEAS ID.
6. **Idempotency re-run** – both cycles run again with no new PIEAS activity; nothing new is created or updated (the single exception being the harmless re-archive of an already-archived record, discussed in Section 6).
7. **Final status** – printed for inspection.

13.1 Final state

Entity	PIEAS rows	OC active	OC archived	Mapping rows	Watermark set?
student	60	60	2	62	yes
faculty	17	17	1	18	yes
course	13	13	1	14	yes

Table 10: Actual `status` output after the full demo run.

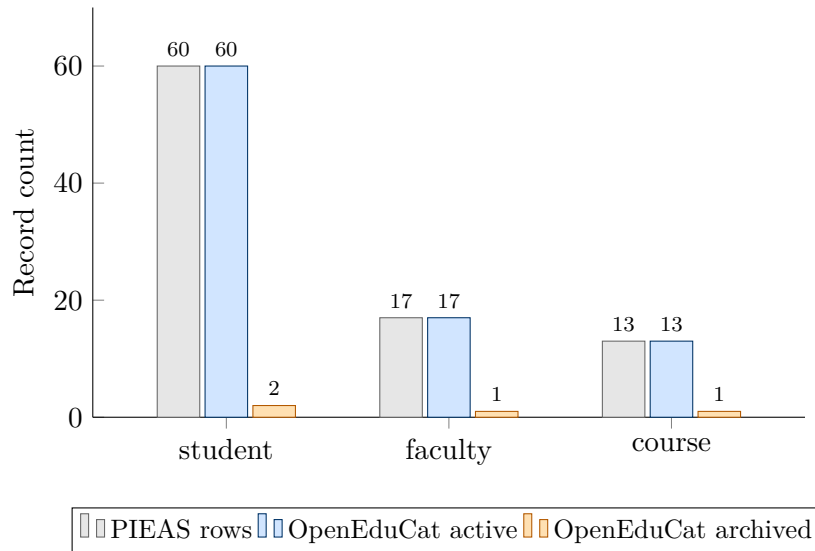


Figure 4: PIEAS row counts vs. OpenEduCat active/archived counts per entity type, after the full demo lifecycle. PIEAS row counts equal OpenEduCat active counts in every case – the sync stayed consistent end to end.

14 Known Characteristics and Limitations

- **Re-archiving a permanently-deleted record.** Discussed in Section 6 – harmless, but worth knowing about when reading deletion-cycle reports.
- **No LLM in the loop.** The Transformer’s field mapping is deterministic Python, not an LLM call. This was a deliberate choice for a synchronization pipeline that must be exactly reproducible and fully unit-testable; an LLM-assisted mapping step could be added later for messier real-world source schemas without changing any other agent.
- **Mock domain filtering.** `OpenEduCatClient.search_read` supports only a flat AND of equality conditions – enough for this pipeline’s needs, not a general Odoo domain evaluator. A real XML-RPC connection supports Odoo’s full domain language; this only matters if the mock client’s surface is extended further.
- **Dummy data, not production data.** PIEAS record volumes, department names, and identifiers are illustrative (drawn from PIEAS’s real academic departments for realism) but entirely synthetic.

15 Path to Production

The system was deliberately built so that pointing it at the real PIEAS database and a real self-hosted Odoo + OpenEduCat instance touches exactly **two files**:

File to change	What changes
<code>pieas_source.py</code>	Replace the SQLite connection with whatever PIEAS’s real database is (or a read-only integration layer), exposing the same <code>fetch_changed</code> / <code>fetch_page</code> / <code>fetch_ids</code> function shapes. <code>ExtractorAgent</code> does not change.
<code>openeducat_client.py</code>	Replace the SQLite-backed methods with real <code>xmlrpc.client.ServerProxy</code> calls against Odoo’s <code>common/object</code> endpoints (<code>execute_kw(db, uid, password, model, method, args)</code>), keeping the same <code>create/write/archive/search_read/read</code> method signatures. <code>LoaderAgent</code> and <code>ValidationAgent</code> do not change.

Table 11: The entire production-swap surface.

Everything else – the Orchestrator’s decision logic, the Transformer’s field mappings, the LangGraph wiring, the CLI, and the test suite’s structure – carries over unchanged, because the whole system was built against the report’s *agent boundaries* rather than against SQLite specifically.

16 Conclusion

The source report set out to design, not build, an agentic multi-agent system solving two coupled problems: a one-time bulk migration and an ongoing, agentic synchronization (including deletion handling) between a source system with no API and a target ERP with a well-documented ORM. This project completes that design into working, tested software: four agents with strictly non-overlapping responsibilities, wired as a single LangGraph state machine that serves all three of the report’s cycles, running against reproducible dummy data through database boundaries that make the report’s ownership rules impossible to violate by accident – verified by 57 passing tests and a one-command, end-to-end demo.

A Appendix: Project File Tree

```

1  src/openedu_orchestrator/
2  config.py           # DB paths, entity types, schedule constants
3  models.py          # pydantic schemas (PieasStudent/Faculty/Course, RunReport,
4  ...                ...)
5  state.py            # LangGraph PipelineState TypedDict
6  pieas_source.py     # dummy PIEAS DB (schema, seeding/mutation/read helpers)
7  openeducat_client.py # mock Odoo/OpenEduCat ORM-shaped client
8  sync_store.py       # sync_mapping / sync_state -- Orchestrator-only
9  seed.py             # deterministic Faker-based dummy data generator
10 graph.py            # LangGraph StateGraph wiring + run_cycle()
11 cli.py              # click CLI
12 agents/
13   extractor.py
14   transformer.py
15   loader.py
16   orchestrator.py
17   validator.py
18 tests/              # pytest suite (57 tests)

```